“It Makes the Code Clearer”: Why Developers Adopt Modern Python Features in Open Source Projects

Walter Mendonça[a,*], Marcos Leite[a], Oseias Romeiro[a], Fausto Carvalho[a], Rodrigo Bonifácio[c], Eduardo Monteiro[a], Gustavo Pinto[b], Paola Accioly[c] and João Saraiva[d,e]

[a] *University of Brasília, Brasília, DF, Brazil*
[b] *Federal University of Pará, Belém, PA, Brazil*
[c] *Universidade Federal de Pernambuco, Recife, PE, Brazil*
[d] *Universidade do Minho, Braga, Portugal*
[e] *HASLab, INESC TEC, Braga, Portugal*

ARTICLE INFO

Keywords:
Empirical software engineering
Mining software repositories
Software evolution
Source code Rejuvenation
Code Comprehension

ABSTRACT

Python has become one of the most widely used programming languages, yet the transition from Python 2 to 3 introduced a tension between innovation and compatibility. While new features such as formatted string literals, type annotations, and structural pattern matching expanded the language’s expressiveness, they also required substantial adaptation of legacy code. Despite the increasing relevance of these features, there is still limited empirical evidence on how modern Python features are being adopted in practice—when developers start using them, how adoption unfolds over time, and what motivations drive these decisions. This paper addresses this gap through a large-scale empirical study of 424 open-source Python projects. Our analysis reveals two distinct adoption patterns: rapid adoption of small syntactic improvements and slower integration of features that require extensive refactoring or ecosystem support. On average, projects begin using with new features within 16 months after their release but take roughly 4 years to achieve broader and sustained adoption. This observation may be partially explained by the transition from Python 2 to 3, which did not preserve full backward compatibility. Complementary qualitative evidence from pull-request discussions indicates that developers are primarily motivated to rejuvenate Python code through improvements in comprehension, safety, and performance, yet often constrained by compatibility requirements and maintenance costs. Together, these findings offer practical insights for tool developers and maintainers seeking to balance innovation and stability in the ongoing rejuvenation of Python source code.

1. Introduction

The Python programming language is among the most popular and versatile languages today, supported by a rich ecosystem that enables developers to build applications across a wide range of domains, including web development, data science, and artificial intelligence [2, 27]. Its clear syntax and extensive library support have contributed to its widespread adoption. This broad reach underscores the strength of the Python community, which actively contributes tools and best practices tailored to diverse application areas, solidifying Python as a cornerstone of modern software development.

Unlike languages that emphasize backward compatibility, such as Java [8, 38] and C++ [33], Python underwent a major breaking change between versions 2 and 3, prompting extensive debate on compatibility and language evolution [36, 37, 27]. At the same time, Python 3 introduced substantial enhancements—native support for asynchronous programming, formatted string literals (f-strings) [28], and new syntax for variable and function type annotations [37]—followed more recently by structural pattern matching, which enables clearer and more concise code [30, 47, 54].

Despite these improvements, the lack of backward compatibility posed significant challenges for Python developers, who needed to adapt legacy code and carefully manage the trade-offs between embracing language innovation and

*Corresponding author
waltimlmm@gmail.com (W. Mendonça); maldsjr@hotmail.com (M. Leite); magalhaes.oseias@aluno.unb.br (O. Romeiro); faustocarva@gmail.com (F. Carvalho); rbonifacio@cin.ufpe.br (R. Bonifácio); edumonteiro@unb.br (E. Monteiro); gpinto@ufpa.br (G. Pinto); prga@cin.ufpe.br (P. Accioly); araiva@di.uminho.pt (J. Saraiva)
ORCID(s):

source code rejuvenation efforts. Rejuvenation is a special type of software maintenance aimed at replacing legacy constructs and idioms of a programming language with their modern counterparts [42]. This leads to a general question: *How, when, and why do open-source Python projects adopt modern language features, and what factors facilitate or hinder the rejuvenation of legacy code?*

Although prior studies have examined the use of specific Python 3 features and language transitions through feature-specific analyses, migration studies, and controlled evaluations [36, 37, 30, 41, 47, 54], existing research remains fragmented in scope and methodology. In particular, there is a lack of systematic, large-scale empirical studies that characterize how modern Python features are adopted over time across open-source projects, especially in the years following the end-of-life of Python 2.7. Moreover, most prior work either focuses on isolated language constructs or relies on static snapshots of feature usage, offering limited insight into when features transition from early experimentation to sustained adoption and how developers' motivations and constraints shape this process. Addressing these gaps can improve our understanding of Python code rejuvenation practices and support developers in making informed decisions about when and how to rejuvenate existing codebases.

To address this gap, we report the results of two empirical studies in this paper. The first investigates the evolution of open-source Python projects to identify adoption trends for 17 modern features introduced in Python 3, from version 3.0 to 3.12. This study includes features previously examined in the literature (such as type annotations) as well as newer features (such as structural pattern matching), for which we provide the first large-scale analysis of their adoption. To understand the motivations and challenges for rejuvenating Python source code, in the second study we review the pull-request comments available in the GitHub repository of the projects we consider in the first study, filtering out those that clearly relate motivations or impediments to rejuvenation efforts. Using thematic analysis, we identified common themes related to motivations (such as improving code comprehension, increasing safety and performance) and challenges (like compatibility issues, maintenance costs, and learning overhead) that might explain the practices for rejuvenating source code in Python. Some of our key findings include:

- **Improving code comprehension drives rejuvenation.** Qualitative findings indicate that enhancing code readability and understanding is the most frequent motivation (53.8% of codes).
- **Compatibility constraints hinder rejuvenation.** Qualitative results show that maintaining compatibility with earlier Python versions and dependent libraries is one of the most frequent challenges.
- **Early use is relatively fast, but generalized adoption lags behind.** While projects often exhibit initial use of new Python features within the first one to two years after release, the transition to generalized and sustained adoption typically unfolds over several years, particularly for features introduced in early Python 3 releases.

Our findings also reveal that some features are widely used in open-source projects—for instance, formatted string literals and type annotations in functions appear in almost 80% and 59% of projects, respectively. By contrast, newer or more specialized constructs, such as structural pattern matching, asynchronous comprehensions, and the `Exception-Group` and `except-*` features, are present in fewer than 5% of projects.

2. The History and Evolution of Python

The Python programming language was created by Guido van Rossum in the late 1980s, with development beginning in 1989 and the first official release—Python 1.0—arriving in 1994 [23, 24]. From the outset, Python distinguished itself through a clean, readable syntax designed to improve code clarity and developer productivity. The language also incorporated key features such as object-oriented programming (e.g., classes and inheritance) and structured exception handling, supporting the development of modular and maintainable software systems. Furthermore, its extensive standard library and smooth integration with other technologies rapidly broadened its adoption, particularly for scripting and automation tasks [37].

The release of Python 2.0 in October 2000 introduced several enhancements, including augmented assignments (e.g., `x += 1`), qualified-name imports, list comprehensions, and a more efficient memory management system, while also marking a period of increased community involvement [36, 37]. This era saw the emergence of key libraries and frameworks such as *NumPy*[1] and *Django*[2], which helped establish Python's strong presence in scientific computing,

[1] https://numpy.org/
[2] https://www.djangoproject.com/

data analysis, and web development. In December 2008, the introduction of Python 3.0 represented a pivotal moment in the language's evolution [37]. Unlike the evolution of other programming languages (e.g., Java), Python 3.0 did not maintain backward compatibility with the Python 2 series, requiring code modifications for migration. The transition aimed to establish a more consistent syntax, improved Unicode support, greater readability, and enhanced robustness. However, as noted by Malloy and Power [37], the lack of backward compatibility posed practical challenges for developers, who often chose to maintain support for Python 2 rather than fully embrace Python 3 features. As a result, many projects limited themselves to the common subset shared between both versions, delaying the widespread adoption of Python's latest advancements.

The Python community and the Python Software Foundation (PSF) have played a crucial role in the language's evolution by organizing events such as PyCon and ensuring a collaborative, transparent development process guided by Python Enhancement Proposals (PEPs) [9]. PEPs are official design documents and the primary mechanism for proposing Python features, collecting community input, and recording design decisions; authors are expected to build consensus around their proposals. Each PEP is discussed in a public, linked thread (e.g., on the Python Discourse via the Discussions-To header), where authors actively gather feedback that will be considered during the review. Standards-Track PEPs pair a precise specification with (ideally) a prototype/reference implementation, so review influences both API design and practical adoption. Under Python's current governance, final acceptance or rejection is decided by the elected Steering Council or a delegated decision-maker. Proposals progress through statuses such as Draft, Accepted, Final, Rejected, or Provisional, which determine when and how new features are introduced and stabilized.

2.1. Evolution of Python 3 Language Constructs

From Python 3.0 (2008) through Python 3.12 (2023), the language syntax evolved through a series of PEP-driven enhancements. In what follows, we outline this evolution chronologically, focusing on changes that directly modified the language grammar while noting releases without syntax updates. This structure helps situate individual features within the broader trajectory toward a more expressive and modern Python language. Python 3.0 (2008) introduced major structural changes to the language grammar. These include keyword-only arguments (PEP 3102), function annotations (PEP 3107), the new `nonlocal` statement (PEP 3104), and extended iterable unpacking (PEP 3132). Developers also gained more control over exception handling with exception context suppression (PEP 409). Together, these features made the language more expressive and flexible, encouraging modern programming practices [10].

Versions 3.1 and 3.2 did not introduce new language features [11, 12]. In turn, Python 3.3 (2012) added syntax for delegating to subgenerators via `yield from` (PEP 380), simplifying generator composition, reducing boilerplate, and enabling more efficient data-processing pipelines [13]. Python 3.4 improved the standard library and the import system, without major changes to the grammar [14]. By contrast, Python 3.5 (2015) introduced additional unpacking generalizations (PEP 448), native coroutine syntax with `async` and `await` (PEP 492) keywords, and the matrix multiplication operator `@` (PEP 465), symplifying the use of asynchronous programming and the implementation of advanced mathematical operations [15]. Python 3.6 (2016) enriched the language with formatted string literals (f-strings, PEPs 498 and 701), asynchronous comprehensions (PEP 530), variable annotations (PEP 526), and asynchronous generators (PEP 525). Collectively, these features improved support for modern applications, particularly in data science and asynchronous programming [16].

Python 3.7 mainly improved performance and internal features, without significant language changes [17]. In 2019, Python 3.8 introduced assignment expressions (PEP 572), also known as the "walrus operator," which enable assignments within conditional statements and loops [18]. In 2020, Python 3.9 allowed direct use of generics in type hints (PEP 585), allowing constructs such as `List[int]` without additional imports. [19]. The following year, Python 3.10 (2021) introduced structural pattern matching (PEPs 634, 635, 636), adding the `match/case` statement and substantially enriching the grammar [20]. In 2022, Python 3.11 introduced exception groups and the new `except*` block (PEP 654) to handle multiple concurrent exceptions, which is especially useful in asynchronous programming [21]. Finally, Python 3.12 (2023) added type parameter syntax (PEP 695), making it easier to define generic type aliases directly in the language, bringing Python closer to languages with advanced static typing [22].

2.2. Source Code Rejuvenation in Python

Source code rejuvenation refers to transforming legacy code to adopt modern language features and idioms [42]. It involves replacing outdated patterns and deprecated constructs with newer, more expressive, and efficient alternatives, ultimately making the code easier to read and maintain [33]. Whereas refactoring is often performed continuously

throughout a project's lifecycle, rejuvenation typically occurs gradually and is more prominent following major language updates.

In Python, this broader migration commonly centers on adopting "Pythonic" idioms that reflect the language's design philosophy. Pythonic idioms are community-accepted patterns and language features that embody Python's characteristic programming style [32, 37, 55]. In the literature, idioms are described as recommended programming styles and features, and in Python they are widely promoted for concise code and, in many cases, improved performance. This culture—often summarized by the "Zen of Python"—encourages developers to prefer idiomatic solutions.

Consequently, Python rejuvenation often entails replacing outdated constructs with more modern, "Pythonic" idioms. Migrating from legacy constructs (e.g., %-based string formatting) to modern features (e.g., f-strings) not only introduces new syntax but also aligns the codebase with contemporary idiomatic standards. For instance, Listing 1 shows how to introduce function annotations in a legacy code. Function annotations, introduced in Python 3.0, enable developers to document expected types directly in the function signature. This change enhances static analysis, tool support, and helps other developers understand code intent, moving the codebase towards better maintainability and compatibility with type checker tools [28].

Legacy style

```
def add(a, b):
    return a + b
```

Modern rejuvenated style

```
def add(a: int, b: int) -> int:
    return a + b
```

Listing 1: Adding function annotations for type hints

Listing 2 illustrates a source code rejuvenation to adopt structural pattern matching—a feature introduced in Python 3.10 that provides a more concise and expressive mechanism for conditional dispatch. By replacing multiple `if/elif` branches with `match/case`, the example makes the intent clearer: each case directly matches the structure of the tuple `cmd` argument and binds the operands accordingly. Moreover, the ordered cases allow exceptional conditions (e.g., division by zero) to be handled explicitly before the general rule, reducing boilerplate, lowering the risk of indexing errors, and improving overall maintainability [54].

In summary, these rejuvenation steps replace verbose, error-prone patterns with clearer idioms, reducing boilerplate and cognitive load while strengthening readability and maintainability.

3. Related Works

The adoption of modern programming language features has been widely studied in software engineering, particularly in the context of software evolution, maintenance, and source code rejuvenation. Prior research consistently shows that the availability of new language constructs does not imply their immediate or uniform adoption, and that adoption is shaped by both technical and social factors. Despite this broad agreement, important gaps remain regarding how adoption is operationalized, how it unfolds over long time spans, and how the transition from early experimentation to widespread use can be systematically identified.

Prior research examines adoption via the acceptance of code modifications, including refactorings or patches that introduce new features. Dantas et al. [7] show that simple transformations are more likely to be accepted than complex refactorings, indicating that cognitive effort and perceived risk affect maintenance choices. Similarly, proactive studies that submit pull requests introducing new language features, such as default methods in Java interfaces, demonstrate that adoption depends not only on technical benefits but also on project governance and maintainer preferences [29]. While these works offer helpful details about decision-making at the moment of change acceptance, they conceptualize adoption as a discrete event and therefore do not capture the gradual diffusion of language features within a codebase.

Another line of research measures adoption based on the prevalence of feature usage in existing systems. Studies on Java Generics show that, even after long periods of availability, usage remains concentrated in specific contexts, revealing partial and uneven adoption [40]. Research on dynamic languages, such as Smalltalk, analyzes the frequency of reflective and dynamic features to assess their impact on static analysis and tool support [4]. Although these studies demonstrate that real-world usage often diverges from language designers' expectations, they typically adopt static or aggregated perspectives and provide limited insight into how usage evolves.

Legacy style

```
def calc(cmd):
    if cmd[0] == "add":
        return cmd[1] + cmd[2]
    elif cmd[0] == "sub":
        return cmd[1] - cmd[2]
    elif cmd[0] == "div" and cmd[2] != 0:
        return cmd[1] / cmd[2]
    elif cmd[0] == "div":
        raise ZeroDivisionError("division by zero")
    else:
        raise ValueError("unknown command")
```

Modern rejuvenated style

```
def calc(cmd):
    match cmd:
        case ("add", x, y):
            return x + y
        case ("sub", x, y):
            return x - y
        case ("div", x, 0):
            raise ZeroDivisionError("division by zero")
        case ("div", x, y):
            return x / y
        case _:
            raise ValueError("unknown command")
```

Listing 2: Dispatching tuple-commands: from chained indexing to structural pattern matching.

Some studies focus specifically on the usage and distribution of Python language features in real-world systems. Peng et al. [41] propose PyScan, an AST-based approach to identify Python features across projects, providing an overview of how different constructs are used in practice. Similarly, large-scale empirical studies analyze the adoption of Python typing-related features, such as type annotations and static typing constructs, revealing uneven uptake across projects and domains [43, 25]. While these works provide valuable snapshots of feature usage, they primarily adopt static or cross-sectional perspectives and do not explicitly characterize how feature adoption evolves over long periods of time.

Complementary research investigates the perceived usefulness and readability of specific Python features through qualitative methods and controlled experiments. For instance, recent user studies on Python structural pattern matching show that developers often prefer this construct over traditional conditional branching in terms of readability and modifiability, although preferences may vary depending on the problem context [54]. Other work explores automated transformations that refactor legacy Python code to adopt structural pattern matching, demonstrating technical feasibility and potential readability gains [47]. However, these studies focus on isolated features and controlled settings, and do not examine whether such features achieve sustained and widespread adoption across open-source ecosystems.

More recent work analyzes adoption as a temporal process by tracking feature usage longitudinally in languages such as JavaScript, TypeScript, and C++. Large-scale empirical studies on JavaScript show that, despite frequent updates to toolchains, the effective use of modern language features often lags behind their introduction and varies substantially across projects [34]. Similarly, studies on TypeScript reveal that upgrading to newer language versions does not necessarily lead to the adoption of newly introduced features [50]. In the C++ ecosystem, analyses of the KDE community report a gradual increase in the use of modern features, often associated with explicit modernization efforts and tool support [33]. Together, these studies establish that adoption is gradual, heterogeneous, and context-dependent;

however, they largely rely on descriptive trend analysis or fixed thresholds, offering limited methodological support to identify when widespread adoption effectively begins.

In summary, prior work demonstrates that language feature adoption is uneven and influenced by multiple factors, but it provides limited means to determine, in a systematic and reproducible way, the onset of general adoption—particularly over long time spans and with sufficient granularity to capture diffusion within projects. This study advances the state of the art by proposing a statistically grounded operationalization of the onset of general adoption based on offline change-point detection applied to long-term time series of feature usage. By analyzing 16 years of Python evolution (2008–2024) and measuring adoption at the file level, our approach reveals that features within the same language may exhibit substantially different adoption regimes, including multi-year delays between their introduction and sustained growth.

Also, in contrast to prior work that either quantifies feature usage statically, evaluates individual language constructs in isolation, or relies on controlled qualitative studies, our work integrates long-term, feature-level adoption analysis with qualitative evidence extracted from real pull request discussions. This combined perspective enables a deeper understanding of not only when Python features begin to generalize, but also why developers choose to adopt, postpone, or avoid them in practice, thereby advancing empirical research on language evolution and source code rejuvenation in software engineering.

4. Study Settings

Our research focuses on understanding how Python developers adopt modern language features in open-source projects. We analyzed 17 Python features introduced from 2006 to 2022 that changed the language's constructs, such as formatted string literals (f-strings), function annotations, and pattern-matching. We specifically examined new syntactic elements of Python, deliberately excluding library-level changes. To guide our investigation, we address the following research questions:

(RQ1) To what extent do open-source Python repositories use modern features?

(RQ2) When did Python developers start adopting modern features?

(RQ3) What are the adoption trends for each modern Python feature?

(RQ4) What are the main motivations that lead developers to rejuvenate legacy Python code, and what are the main challenges that prevent them from doing so?

Answering the first research question helps us understand how much Python developers adopt modern language features and whether recent Python releases help develop Python applications. Python developers often prefer features that make their code simpler [41], so we expect wide adoption of easy-to-use syntax like f-strings. On the other hand, developers might not widely use features such as Structural Pattern Matching or Assignment Expression (walrus operator) because they already rely on familiar solutions.

Understanding these differences shows why it is important to study how open-source Python projects use each modern feature. The second research question examines when developers begin using new Python features and whether they feel confident moving away from familiar practices toward newer ones. By examining the third research question, we aim to identify if there is a growing trend in adopting modern Python features among open-source projects. The fourth question focuses on the motivations for Python code rejuvenation and examines the challenges developers encounter that hinder them from rejuvenating legacy systems.

Our research was organized into two distinct studies. The initial study utilized descriptive statistics and time series analysis to examine the adoption and trends of modern Python features. Initially, we outline the criteria used to select the 17 contemporary features of the Python language for examination. We detail the procedure employed to select the open-source repositories that formed our dataset. This section outlines the study's methodology, specifying the tools and methods utilized for data collection and analysis in addressing research questions RQ1, RQ2, and RQ3. Next, we employed thematic analysis [5] to elucidate the motivations and challenges associated with Python code rejuvenation. In the second study, we gathered data from code review comments to address our final research question (RQ4). We have made our replication package [44] accessible to the public. It encompasses our datasets, the scripts for analysis, and all the materials to replicate the findings of our study.

Table 1
Python Features, PEPs, Versions, Release Date, and Examples (ordered by Python version and PEP number).

Feature	Python Version	PEP(s)	Release Date	Simple Example
Keyword-only Arguments	3.0	3102	2008-12	`def func(*, arg): pass`
`nonlocal` Statement	3.0	3104	2008-12	`def outer(): nonlocal x; x = 10`
Function Annotations	3.0	3107	2008-12	`def func(x: int) -> int: return x`
Extended Iterable Unpacking	3.0	3132	2008-12	`a, *rest, b = range(5)`
Syntax for Delegating to Subgenerator (Yield From)	3.3	380	2012-09	`yield from subgenerator()`
Suppressing Exception Context	3.3	409	2012-09	`try: 1/0 except ZeroDivisionError: raise ValueError('Invalid') from None`
Additional Unpacking Generalizations	3.5	448	2015-09	`{**{'a':1}, **{'b':2}}`
Matrix Multiplication Operator (`@`)	3.5	465	2015-09	`a @ b`
Async and Await Syntax	3.5	492	2015-09	`async def foo(): await bar()`
Formatted String Literals (f-strings)	3.6	498,701	2016-12	`f"Hello, {name}"`
Asynchronous Generators	3.6	525	2016-12	`async def gen(): yield x`
Variable Annotations	3.6	526	2016-12	`x: int = 10`
Asynchronous Comprehensions	3.6	530	2016-12	`[i async for i in aiter()]`
Assignment Expressions	3.8	572	2019-10	`if (squared := num ** 2) > 10`
Type Annotations Generics	3.9	585	2020-10	`x: list[int] = [1, 2, 3]`
Structural Pattern Matching	3.10	634,635,636	2021-10	`match x: case 0: print("zero")`
Exception Groups and `except*`	3.11	654	2022-10	`try: raise ExceptionGroup('', [ValueError()]) except* ValueError: pass`
Type Parameter Syntax	3.12	695	2023-10	`type ListOrSet[T] = list[T] | set[T]`

4.1. Features Selection

Table 1 lists the modern Python features analyzed in our study. The table includes the Python version that introduced each feature, the associated Python Enhancement Proposals (PEPs), their official release dates, and simple code examples illustrating their use. Although our selection does not cover four out of the 21 modern Python 3.x features that modified the language syntax, it still provides a comprehensive view of Python's evolution.

We excluded certain Python features from our analysis due to limitations in detecting their usage through static analysis alone. Specifically, we removed "Underscores in Numeric Literals" because the Python AST library requires converting numeric nodes back into source code strings, leading to frequent exceptions and false positives during analysis. Similarly, we excluded "Postponed Evaluation of Type Annotations" as this feature delays evaluating type annotations until runtime, making it difficult to detect reliably through static analysis.

Additionally, we omitted "Relaxed Grammar Restrictions on Decorators" since decorators have existed since Python 2, complicating the identification of differences introduced exclusively from Python 3 onward. Lastly, we did not consider the "Union Operators for dict" feature, as static analysis cannot reliably track cases where variables initially defined as dictionaries might later reference different types at runtime, leading to potential inaccuracies in detection.

4.2. Dataset Selection

Our method for selecting projects was inspired by the approach of Zhao et al. [56]. First, we searched GitHub repositories that use Python as their main language, created before 2012, and still active in 2024. We chose projects at least 10 years old to examine mature and legacy systems, as older systems are more likely to face challenges related to outdated technologies and coding practices [46]. This decision helps us study projects that transitioned from Python 2 to Python 3, which may have required source code rejuvenation to stay relevant and active.

We focused our analysis on active repositories, identified using the SEART tool [6]. Initially, we gathered 1,199 repositories and excluded forks, tutorials, and repositories with fewer than 1,000 commits, reducing the list to 460 projects. Finally, we excluded 36 repositories due to data-collection issues. The resulting dataset in Table 2 comprises 424 Python projects that together amount to approximately 56M lines of code, 1.45M stars, and 357K forks, with a median of 753 stars, 219 forks, and 69 contributors per project, indicating a set of mature and actively maintained repositories.

The dataset covers a broad range of domains within the Python ecosystem, including data science and machine learning, web development, scientific computing, developer tooling, and general-purpose libraries. Most projects are libraries or frameworks intended for reuse, alongside a smaller number of end-user applications. Well-known and widely adopted projects such as Flask, scikit-learn, Requests, and pandas illustrate the prominence of domains where language evolution and long-term maintenance pressures are particularly relevant.

Table 2
Dataset characterization of the 424 analyzed Python projects.

Characteristic	Value
Projects	424
Total lines of code (LOC)	~56M
Median LOC per project	35,561
Median contributors per project	69
Median repository size (files)	15,275
Total stars	~1.45M
Median stars per project	753
Total forks	~357K
Median forks per project	219
Project types	Libraries, frameworks, applications
Main domains	Data science, ML, web, DevOps, utilities

4.3. Data Collection and Analysis Procedures

In the first study, we analyzed the source code history of 424 GitHub projects to investigate the utilization and adoption patterns of modern Python features. To accomplish this, we built PyMiner, a tool intended to analyze the source code history of selected projects and collect data regarding specific Python features. PyMiner uses Python's built-in AST library to parse Python files, generate abstract syntax trees (AST), and traverse the Python code using visitors.

Our analysis pipeline uses PyMiner to inspect Python projects from January 1, 2008, to December 31, 2024. First, we compiled a list of Python repositories from GitHub using the SEART tool [6], then cloned these repositories locally. PyMiner then reviews each repository's commit history at 30-day intervals, reducing the total number of commits to be analyzed. For each selected commit, PyMiner retrieves the project state and traverses Python files to collect information on modern features. The results for each project are stored in individual CSV files, which we later combine into a single file for analysis.

We used this consolidated dataset to address our first three research questions (RQ1, RQ2, and RQ3). For RQ1, we conducted a descriptive analysis to measure how often each feature was used and calculated the median number of files per revision containing these features. To answer RQ2, we identified when each modern Python feature first appeared in the repositories. Lastly, for RQ3, we applied a time-series analysis, using the timing and the count of files with occurrences data of each feature to model their adoption trends. All analyses were carried out using Python scripts.

In the second study, to address RQ4, we developed a Python-based infrastructure that leverages the GitHub API to scrape comments from pull requests, automating the data collection process. This infrastructure iterates over each pull request (PR) in the repositories in our dataset and retrieves the related PR comments. In this study, we mined 395,702 pull request comments and populated a database for analysis.

Next, we employed a three-stage filtering process to identify pull request comments pertinent to source code rejuvenation in Python projects. Initially, a large corpus of comments (3,565) was mined using a curated set of keywords derived from Python language features introduced in recent PEPs, including "asynchronous comprehensions", "formatted string literals (f-strings)", "structural pattern matching", and "assignment expressions", as well as specific Python Enhancement Proposals (PEPs) like PEP 492, PEP 498, and PEP 634. Additionally, the query incorporated common variations and related terms—such as "async def", "fstring formatting", "match-case statements", and "walrus operator"—to capture a broader range of relevant discussions.

Subsequently, the filtered comments underwent a second-level screening using a simple prompt-based classification with ChatGPT (gpt-4o). The prompt used in this study (available in the Zenodo repository [44], file *source_code_rejuvenation_search.py*) asked the model to decide whether each comment pertained to source code rejuvenation—defined as updating legacy code with modern Python idioms—and to provide a brief summary when relevant.

To assess whether the LLM-based filtering stage systematically excluded relevant discussions, we conducted a manual validation of a random sample of comments classified as unrelated. We randomly selected 100 comments from the approximately 3,000 comments rejected by the LLM and manually inspected them to identify potential false negatives. The majority of the sampled comments were clearly unrelated to source code rejuvenation, focusing instead

on localized design decisions, documentation concerns, or implementation details without reference to legacy code or language evolution. For example, a comment discussing whether expressions of default arguments and annotations should be collapsed in documentation—"This change has pros and cons… there might be a case for collapsing the expressions of default arguments and keeping the expressions of annotations"—was classified as irrelevant because it concerns how existing language features are represented and implemented, rather than any effort to rejuvenate legacy code.

Only a small subset of the sampled comments (approximately 5%) could be considered potential false negatives. These cases typically involved implicit or weakly articulated signals related to modern language features that did not explicitly frame their use as part of a rejuvenation effort. For instance, a comment such as "Are f-strings supported in lambdas?" reflects a clarification or doubt about the applicability of a modern Python feature, but does not express a motivation to replace legacy constructs, nor does it describe challenges or impediments to modernization. Overall, this validation indicates that the LLM-based filtering did not systematically exclude rejuvenation-related discussions, while substantially reducing the manual analysis effort.

This lightweight step complements the keyword filter and scales semantic screening to large volumes while reducing manual effort, consistent with evidence that prompt-based LLMs can perform effective text classification in low-resource settings and capture task-relevant cues via prompts [35, 52]. This process yielded 494 pull request comments, which were then manually examined in a third phase using thematic analysis. The thematic analysis was conducted manually by the authors to ensure interpretive depth.

Following, we adopted a thematic analysis approach, following Saldaña's guidelines for qualitative data analysis [49]. We began with open coding, an initial phase in which relevant text segments are identified and associated with descriptive labels or tags that reflect emerging concepts and themes within the data. Next, we proceed to classification, systematically grouping the initial codes into broader categories based on shared characteristics. Throughout this process, we continuously refined and revisited both codes and categories as new data and insights emerged.

The data analysis procedure was conducted in two cycles to verify the relevance and accuracy of the selected pull request comments. Three authors collaboratively conducted the thematic coding process, with the first having prior experience in qualitative software engineering research. During the initial cycle, the first and third authors of this paper examined the 494 code review comments identified through the textual query, distinguishing those pertinent to motivations and challenges for rejuvenation efforts from those that merely cited the name of modern Python features. From this process, 77 quotes from comments were identified as potentially relevant.

During the second cycle, the second and third authors conducted independent reviews of each other's work, noting areas of agreement and disagreement regarding the coding. In cases of disagreement, the authors, along with an additional author, discussed the coding until a consensus was reached. The first author of this paper categorized the codes into thematic groups. In Section 6, we describe and present our findings.

5. First Study: Prevalence and Adoption Trends of Modern Python 3 Features

This section reports a quantitative analysis of the adoption of modern *Python* features. By traversing the complete revision history of each repository we counted every occurrence of the 17 features listed in Table 3.

5.1. Prevalence Assessment

Table 3 highlights that *function annotations* stand out as the most prevalent modern Python feature: they appear 306,196 times, and are present in nearly 59% of the analyzed projects. This extensive adoption reflects their central role in enabling static type checking and facilitating gradual typing through tools such as `mypy`. While formatted string literals (f-strings) remain highly popular—appearing 124,312 times and present in 79% of analyzed projects —the sheer volume of function annotations underscores their fundamental importance in the evolution of Python's type system and modern development practices. In total (considering the last commit of all projects), our dataset contains 1,144,365 traditional (non-annotated) functions and 135,171 annotated functions, meaning that only about 10% of all functions include at least one type annotation. This disproportion illustrates that, although annotations are widespread across projects, their internal coverage remains limited—supporting the findings of Grazia and Pradel [25] that annotations are still sparsely distributed within individual codebases. Similarly, *variable annotations* (84,853 occurrences, 52.6% of projects) follow a comparable pattern of gradual integration since Python 3.6.

Table 3
Project–level adoption of modern Python features. Δ Months = (First Occurrence) − (Release month from Table 1); negative values indicate pre-release (alpha/beta) occurrences.

Feature	Total of Occurrences (#)	Projects Adoption (%)	First Occurrence	Δ Months
Formatted String Literals (f-strings)	124312	79.71	2016-05	-7
Function Annotations	306196	58.96	2012-05	41
Variable Annotation	84853	52.59	2017-01	1
Keyword-only Arguments	49622	47.16	2011-11	35
Additional Unpacking Generalizations	4029	42.92	2015-09	0
Yield From Expression	2093	36.08	2012-05	-4
Assignment Expression	2772	23.58	2019-12	2
Nonlocal Statements	586	20.28	2010-12	24
Coroutines (async and await syntax)	39496	19.57	2015-12	3
Extended Iterable Unpacking	416	19.33	2009-12	12
Suppressing Exception Context	933	17.68	2015-03	30
Matrix Multiplication	2304	8.25	2018-05	32
Structural Pattern Matching	2054	4.48	2021-05	-5
Asynchronous Comprehensions	359	2.59	2019-11	35
ExceptionGroup	11	0.70	2022-05	-5
Type Parameter Syntax	5	0.47	2023-07	-3
Except (*)	1	0.23	2022-05	-5

Keyword-only arguments and the generalised unpacking operators also display moderate diffusion, each reaching more than 40% of projects. In contrast, several features have yet to gain traction. Exception groups (PEP 654) and the type-parameter syntax (PEP 695) appear in fewer than 1% of projects, while structural pattern matching (Python 3.10) is present in only 4.5% of repositories. Conversely, we did not observe evidence of the use of Asynchronous Generators that were in our initial subset.

Table 4 presents summary statistics for the occurrences of various Python features in GitHub projects. Each feature is listed with its median, mean, standard deviation (std), maximum (max), and minimum (min) number of occurrences across the analyzed repositories. Feature usage varies considerably across projects. For example, Formatted String Literals (f-strings) exhibit a median of 31 occurrences per project, while Variable Annotations and Function Annotations present medians of 1 and 3, respectively. In contrast, most other features display a median of zero. The distribution is also highly uneven, as reflected by the large standard deviations. For instance, in the *sentry* project, Function Annotations reach a maximum of 26,605. We also observe a similar lack of uniformity in the usage of Variable Annotations and f-strings across projects.

Answer to RQ1

Our results show that modern Python features are adopted unevenly. *Function annotations* dominate in absolute frequency with more than 306,000 occurrences, while other widely used constructs include *f-strings*, present in nearly 80% of projects. Several features, such as *variable annotations*, *keyword-only arguments*, and *additional unpacking generalizations*, appear in around half of the projects, whereas recent constructs like *structural pattern matching* and *exception groups* remain rare.

We also compare each feature's official release date in the Python language with the date of its first occurrence in our dataset. In our analysis, negative Δ values indicate the adoption of language features from pre-release (alpha or beta) versions of Python prior to their official stable release. Interestingly, some features appear earlier than their official final release dates. The first case is *formatted string literals (f-strings)*, which officially became part of Python 3.6 in December 2016, but whose first appearance in our dataset was in May 2016. This aligns with the release of Python 3.6.0a1, the first alpha version, published on May 17, 2016 (Python 3.6.0a1, Python 3.6 Release Schedule). The second case is the *yield from expression*, formally introduced with Python 3.3 in September 2012, but observed in our dataset as early as May 2012. This corresponds to the alpha releases of Python 3.3, such as 3.3.0a3 (May 1, 2012) and 3.3.0a4 (May 31, 2012), which already included support for PEP 380 (PEP 380, Python 3.3.0, Python 3.3 Release Schedule). Furthermore, other features that appear before their release include structural pattern matching,

Table 4
Summary statistics for the occurrence of Python feature in GitHub projects.

Feature	Min	Median	Mean	Max	Std
Function Annotations	0	3	361.08	26605	1452.81
Formatted String Literals (f-strings)	0	31	293.19	13188	966.48
Variable Annotation	0	1	200.13	24599	1332.99
Keyword-only Arguments	0	0	58.52	3497	291.56
Coroutines (async and await syntax)	0	0	23.29	4170	213.73
Assignment Expression	0	0	6.54	229	27.52
Matrix Multiplication	0	0	5.43	924	49.69
Yield From Expression	0	0	4.94	289	19.80
Suppressing Exception Context	0	0	2.20	265	15.09
Additional Unpacking Generalizations	0	0	1.58	523	13.39
Nonlocal Statements	0	0	1.38	64	5.88
Extended Iterable Unpacking	0	0	0.98	45	4.07
Structural Pattern Matching	0	0	0.54	345	9.25
ExceptionGroup	0	0	0.03	5	0.33
Type Parameter Syntax	0	0	0.00	3	0.09
Except (*)	0	0	0.00	1	0.05
Asynchronous Comprehensions	0	0	0.28	248	7.10

ExceptionGroup/except*, and Type Parameter Syntax. These findings show that developers sometimes adopt new language features directly from alpha or beta versions, well before they are available in stable releases.

When comparing all the features in our dataset, we observe different timelines between their official release dates and their first appearance. *Function annotations* were released in December 2008 with Python 3.0, but the first use in our dataset occurred in May 2012. *Keyword-only arguments*, also introduced in December 2008, first appeared in November 2011. *Nonlocal statements* were released in December 2008 and first appeared in December 2010, while *extended iterable unpacking*, from the same December 2008 release, appeared in December 2009. *Suppressing exception context* was added in September 2012 with Python 3.3, and first appeared in March 2015. In contrast, other features appeared soon after their official releases: *variable annotations* released in December 2016 first appeared in January 2017, *assignment expressions* released in October 2019 appeared in December 2019, and *coroutines (async/await)* released in September 2015 appeared in December 2015. Some features were observed immediately: *additional unpacking generalizations*, released in September 2015, first appeared in the same month.

Answer to RQ2

Our results show that modern Python features have been adopted at very different rates. Some features (e.g., variable annotations, assignment expressions, async) were adopted quickly, while others (e.g., function annotations, keyword-only arguments, `nonlocal`) saw slower adoption. We also observe that certain features—including f-strings, `yield from`, structural pattern matching, and ExceptionGroup/`except*`—appeared in projects even during alpha or beta releases, indicating early experimentation among some developers.

5.2. Adoption Trend Assessment

Although a feature may appear in repositories shortly after its official release, substantial growth in adoption often occurs later. Here, we analyze the delay between initial use and the onset of adoption growth for each feature. We analyze monthly commit time series spanning from 2008 to 2024 to identify adoption patterns. For each month in this period, and for each project and feature, we counted the number of Python source files containing at least one occurrence of the feature, considering all Python files present in the commit analyzed for that month. These per-project counts were then aggregated across all projects to obtain a monthly, dataset-level time series of file-level usage for each feature. Our analysis considers only features present in at least 10% of the projects in the dataset.

To identify the onset of broader adoption of Python language features, we analyze monthly time series of feature usage using offline change point detection. Specifically, we apply the PELT algorithm with an ℓ_2 cost function as implemented in the `ruptures` library [53] to identify candidate structural breakpoints in each time series. Prior to

detection, raw occurrence counts are transformed using $\log(1 + x)$ to mitigate heteroscedasticity commonly observed in count-based temporal data. Change points are detected by minimizing a penalized segmentation objective, where the penalty is deterministically derived from the series length and variance, ensuring reproducibility and avoiding manual parameter tuning.

For each candidate change point b, we assess whether it corresponds to the onset of generalized adoption by comparing robust central tendencies before and after b within a symmetric temporal window H. Specifically, we compute the median feature usage in the pre-onset window $[b-H, b)$ and the post-onset window $[b, b+H)$, and define an adoption score as:

$$\text{score}(b) = \frac{\text{median}\left(y_{b:b+H}\right) - \text{median}\left(y_{b-H:b}\right)}{\max(1.4826 \cdot \text{MAD}(y),\ \varepsilon)},$$

The scale is given by $1.4826 \cdot \text{MAD}(y)$, where $\text{MAD}(y) = \text{median}(|y - \text{median}(y)|)$ denotes the Median Absolute Deviation, a nonparametric and outlier-resistant measure of dispersion [51, 45, 26]. The constant 1.4826 ensures consistency with the standard deviation under normality. To ensure numerical stability and avoid inflated scores in near-constant series, the denominator is lower-bounded by a small constant $\varepsilon = 10^{-3}$. This formulation yields a robust, dimensionless effect size that captures sustained shifts in feature usage while mitigating the influence of noise, outliers, and early sporadic adoption.

When comparing the release of each feature with the onset of generalized adoption trends, we observe substantial delays for features introduced in Python 3.0. *Keyword-only arguments*, released in December 2008, reached their adoption onset only in February 2012, corresponding to a delay of approximately 3.2 years. Similarly, the *nonlocal* statement, also released in December 2008, exhibited adoption onset in October 2012, after about 3.8 years.

Function annotations, despite being introduced in Python 3.0, entered a generalized growth phase only in October 2016, resulting in a markedly longer delay of nearly 7.8 years. *Extended iterable unpacking* followed a comparable long-term pattern, with adoption onset observed in September 2019, almost 10.8 years after its release. This slow and staggered adoption aligns with the findings of Di Grazia and Pradel [25], who reported that, despite steady growth since 2017, the overall coverage of function annotations remained low—only about 8% of function arguments and return types were annotated in 2021.

Together, these results indicate that although several Python 3.0 features are present in a non-trivial fraction of projects (e.g., function annotations appear in nearly 59% of projects in our dataset), their integration into individual codebases unfolds as a gradual rejuvenation process, characterized by incremental and localized adoption over extended periods.

For Python 3.3, adoption delays are noticeably shorter. The *yield from* expression, released in September 2012, reached its adoption onset by May 2014, approximately 1.7 years later. Likewise, *suppressing exception context*, introduced in the same release, entered its growth phase in January 2020, reflecting a longer but still moderate delay of about 7.3 years. Python 3.5 features display heterogeneous adoption dynamics. *Coroutines* (`async`/`await`), released in September 2015, reached generalized adoption by September 2016, after only 1 year. In contrast, *additional unpacking generalizations* required around 2.6 years, with growth beginning in April 2018.

Features introduced in Python 3.6 generally exhibit faster adoption. *Formatted string literals (f-strings)*, released in December 2016, reached widespread growth by January 2019 (about 2.1 years), while *variable annotations* followed closely, with adoption onset in May 2018 (approximately 1.4 years). Finally, among later features, *assignment expressions* (Python 3.8) show one of the shortest delays: released in October 2019, they reached adoption onset by January 2021, after only 1.3 years.

Several inflection points observed in our adoption curves temporally align with major ecosystem-level events in the Python community. For example, Python 2 officially reached its end-of-life on January 1, 2020, after which it no longer received security patches or official support, effectively removing Python 2 as a viable long-term platform and increasing pressure for migration to Python 3.[3] This transition was reinforced by key ecosystem decisions, including the release of Django 2.0 in December 2017, which dropped support for Python 2 entirely and required Python 3, as well as the coordinated phase-out of Python 2.7 support by major scientific libraries such as NumPy, aligned with the Python core team's deprecation timeline.[4][5] In parallel, the maturation of the Python tooling ecosystem between 2016

[3] https://www.python.org/doc/sunset-python-2/
[4] https://docs.djangoproject.com/en/2.0/releases/2.0/
[5] https://numpy.org/neps/nep-0029-deprecation_policy.html

and 2019 — particularly the emergence and consolidation of static type checking tools such as `mypy`, which builds on function annotations — lowered the practical cost of adopting modern language features.[6] While these events do not establish causality, their temporal proximity to the observed acceleration in feature adoption suggests that external, ecosystem-driven pressures played an important contextual role in reducing long-standing inertia and enabling broader source code rejuvenation.

Answer to RQ3

Our results indicate that modern Python features require, on average, approximately **4 years** to reach the onset of generalized adoption, although this process exhibits substantial variability across language versions and feature types. Adoption delays range from as little as **1–1.5 years** for features such as *coroutines* (`async/await`), *variable annotations*, and *assignment expressions*, to nearly **11 years** for *extended iterable unpacking*, a feature introduced in Python 3.0. Early Python 3.0 features consistently display longer maturation periods—often exceeding **7 years**—suggesting prolonged inertia in legacy codebases. In contrast, features introduced from Python 3.5 onward typically reach generalized adoption within **1–3 years**, revealing a clear acceleration in uptake over Python's evolution. The temporal alignment of these adoption inflection points with ecosystem-level events, such as the Python 2 end-of-life and the consolidation of Python 3–only frameworks and tooling, suggests that generalized adoption is best understood as a socio-technical rejuvenation process shaped not only by language design, but also by external pressures that reduce postponement and facilitate modernization.

6. Second Study: Motivations and Challenges for Rejuvenating Python Code

In order to address our fourth research question, we conducted a manual analysis of a subset of code review comments from the repositories examined in our first study to gain insight into the motivations and challenges developers encounter when rejuvenating Python code. In the scope of this research, code review is a collaborative software engineering practice where developers systematically examine each other's code to improve quality, correctness, and maintainability [39, 3, 31]. Each pull request comment was treated as an individual review unit, and the excerpts reported in this paper correspond to substrings of these comments selected for their relevance to rejuvenation discussions. Themes were derived inductively based on conceptual relevance rather than frequency, although the number of codes per category is reported for transparency.

6.1. Motivations

Our qualitative analysis of developer discussions in pull requests identified a set of recurring motivations that drive Python code rejuvenation efforts. These motivations reflect not only the pursuit of cleaner and more maintainable code, but also the desire to adopt modern language features that enhance safety, and performance. Each motivation was grouped into a specific subcategory, capturing a distinct aspect of why contributors decide to conduct source code rejuvenation efforts. In the following paragraphs, we describe each subcategory, report its frequency in the analyzed dataset, and illustrate it with representative comments from developers, highlighting both the feature being adopted and the intended benefit for the project.

Improve code comprehension This subcategory refers to changes aimed at making code easier to read and understand, especially during reviews and maintenance. It appears 37 times in the analyzed pull request comments, capturing its role in shaping how source code rejuvenation unfolds in Python open-source projects. For example: *"Consider adding type hints for better code clarity and to leverage static type checking."* (`materialsproject/pymatgen` • `PR #3758`), *"Use f-strings for more readable and efficient string formatting."* (`materialsproject/pymatgen` • `PR #3785`), and *"I much prefer the f-string equivalent as much easier to read."* (`pylons/ pyramid` • `PR #3701`). These examples show how contributors use modern constructs such as f-strings and type annotations to make the intent of the code more clear, readable and easier to read. Improving code comprehension reduces the mental effort required for understanding logic during reviews and maintenance, allowing both current and future developers to reason about the code with less cognitive load.

[6] `https://mypy-lang.org/`

Improve safety This subcategory focuses on changes that reduce the likelihood of runtime errors and improve the robustness of the system. It appears 7 times in the analyzed pull request comments. For example: *"This function should be `async def` and the execute commands should `await`. This way, if someone tries to await a function that raises an error, a second error won't pop up saying you tried to await an exception."* (`redis/redis-py` • `PR #2099`) and *"Maybe these should all become keyword-only arguments for safety, meaning an error if they try to pass the `slice_order` in by position, breaking backward compatibility and warning them of the problem."* (`nipy/nipy` • `PR #266`). These comments reflect the importance of aligning code with safer execution patterns, including keyword-only arguments to prevent accidental misuse, and asynchronous functions to avoid blocking I/O. Such measures reduce the risk of subtle runtime errors and make systems more robust.

Improve performance This subcategory focuses on rejuvenation efforts motivated by potential gains in execution speed or resource efficiency. It appears a time in the analyzed pull request comments. For example: *"f-strings and percent formatting are marginally faster than the ".format() too."* (`pylons/pyramid` • `PR #3701`). These observations illustrate that even marginal gains from features like `f-strings` and % formatting can matter in performance-critical paths. Contributors weigh these potential benefits against clarity and maintainability, reflecting a pragmatic approach to conduct rejuvenation efforts in the Python community.

6.2. Challenges

Each challenge was grouped into a subcategory that captures a distinct obstacle faced by developers. In the following paragraphs, we present each subcategory and illustrate it with representative comments, highlighting the tension between adopting modern Python features and maintaining project stability.

Compatibility issue This subcategory refers to constraints from supporting specific Python versions or environments, which can delay or block modern features. It appears **5** times in the analyzed pull request comments. For example: *"X | Y syntax for unions requires Python 3.10."* (`jazzband/tablib` • `PR #516`) and *"But until Python 3.6 support is dropped, we can only add annotations which are Python 3.5 compatible."* (`twisted/twisted` • `PR #1294`). These cases show how maintainers must align rejuvenation efforts with version support policies, often deferring the use of features like f-strings or advanced type annotations until older versions are no longer supported, ensuring the code remains functional across its intended deployment range.

Maintenance cost This subcategory refers to the increasing workload and complexity that modern language features can impose on testing and maintenance processes. It appears two times in the analyzed pull request comments. For example: *"If we're going to be eventually using keyword-only arguments in many places in the library, the complete tests could become onerous."* (`networkx/networkx` • `PR #6267`) and *"Thank you, @contributor! I implemented this, but I agree that this won't scale well in general if keyword-only arguments are more widely used."* (`networkx/networkx` • `PR #6267`). These examples show that the introduction of features such as keyword-only arguments can increase the combinatorial burden of testing and the overall maintenance effort. Developers often delay widespread adoption of such features until their usage patterns, testing strategies, and supporting infrastructure can handle the added complexity efficiently.

Learning overhead This subcategory reflects the perception that certain rejuvenation efforts may introduce complexity that outweighs their immediate benefits, particularly for contributors less familiar with newer language constructs. It appears 1 time in the analyzed pull request comment. In the context of PEP 465, which proposed adding the `@` and `@=` operators for matrix multiplication, a contributor expressed: *"[Q1] It provides no balanced view of the status quo. [Q2] You need to convince those outside the NumPy community that the addition of two new operators is justified for all Python users, most of whom are not users of linear algebra. [Q3] It adds to the Python learning curve with no benefit to most and very limited benefit to the users of linear algebra."* (`numpy/numpy` • `PR #4351`). The developer suggest that even well-intended syntax can be seen as overhead, which can slow the adoption of rejuvenation efforts.

Answer to RQ4

Python source code rejuvenation is mainly driven by goals such as improving code comprehension, safety, and performance often through modern features like `f`-strings, type annotations, keyword-only arguments, `async`/`await` patterns, and advanced typing annotations. Key challenges include version compatibility issues, high maintenance costs, and the learning curve of new features. Effective rejuvenation balances these benefits and obstacles to rejuvenate the source code without compromising stability or accessibility.

7. Implications

Our findings highlight several important aspects regarding the adoption of modern Python features across open-source projects, combining large-scale quantitative evidence with qualitative insights derived from developers' discussions in pull request comments.

For **RQ1**, the results reveal substantial variability in the adoption rates of modern Python features. Constructs that provide immediate and tangible benefits to code readability and comprehension—such as *formatted string literals (f-strings)*, *function annotations*, and *variable annotations*—achieved broad adoption across projects. In contrast, features that introduce new programming paradigms or require more substantial conceptual shifts, including *structural pattern matching*, *asynchronous comprehensions*, and *exception groups (except*)*, exhibit slower and more fragmented adoption. Qualitative evidence from pull request discussions provides contextual explanations for these patterns, as some developers explicitly mention concerns related to learning effort, refactoring scope, and uncertainty about long-term benefits when evaluating such features.

Regarding **RQ2**, our analysis shows that developers typically begin experimenting with new Python features within one to two years after their official release. Evidence of feature usage often appears shortly after release, indicating that initial adoption tends to occur relatively early, even if usage remains limited in scope at this stage. This pattern is observed across multiple features and suggests a general willingness within the Python ecosystem to explore new language constructs soon after they become available.

In addition, some features appear in repositories even before their stable release, including *f-strings*, *yield from*, *structural pattern matching*, and *ExceptionGroup/except**. Qualitative observations indicate that such early experimentation is typically confined to a small subset of projects and is often accompanied by discussions related to feature maturity, documentation, or potential impact on existing code, reflecting cautious exploration rather than immediate, widespread use.

The analysis of adoption trends (**RQ3**) further shows that, although initial usage often emerges shortly after release, widespread and stable adoption typically takes several years to materialize. On average, modern Python features require approximately **four years** to transition from early experimentation to sustained growth across projects. Lightweight syntactic enhancements tend to diffuse more rapidly, whereas features that demand broader architectural changes or deeper language understanding exhibit longer maturation periods. Developers' comments occasionally reference compatibility constraints—particularly support for older Python versions—as a factor that may delay broader adoption.

Finally, with respect to **RQ4**, our qualitative analysis reveals that source code rejuvenation in Python is primarily motivated by the desire to improve code comprehension, followed by concerns related to safety and correctness. Developers commonly adopt features such as `f`-strings, type and variable annotations, and keyword-only arguments to enhance clarity and reduce ambiguity. At the same time, rejuvenation efforts face notable impediments, including compatibility with legacy environments, the maintenance cost of large-scale changes, and the learning overhead associated with newer language constructs. These qualitative findings complement the quantitative results by providing contextual explanations for why certain features, in many cases, exhibit prolonged adoption delays despite their potential benefits.

Overall, our results suggest that, in open-source settings, successful rejuvenation strategies require balancing the immediate gains of adopting modern language features with the practical constraints imposed by project scale, version support policies, and developer expertise. By combining longitudinal adoption analysis with qualitative insights from real-world developer discussions, this study provides a more nuanced understanding of how and why Python features transition from experimentation to general use in open-source software ecosystems.

8. Threats to Validity

Our study has severeal limitations. A few large or highly active repositories could disproportionately influence aggregate trends—for example, the `sentry` project shows unusually high counts of `f`-strings and annotations. To reduce this bias, we analyzed the *monthly number of files with at least one use* of each feature instead of raw counts and applied smoothing to limit short-term spikes. Although large-scale adoptions within a single project may still affect the curves, they accurately reflect real repository-wide changes. RQ1 and RQ2 provide complementary project-level views that are less sensitive to these effects, and future work could further normalize by project size or count projects using each feature.

Our operationalization of *generalized adoption onset* relies on detecting sustained shifts in the central tendency of monthly feature usage time series. To mitigate sensitivity to outliers, early sporadic usage, and heavy-tailed count distributions, we employ robust statistics throughout the analysis. In particular, adoption strength is quantified using the difference between pre- and post-onset medians, normalized by the Median Absolute Deviation (MAD), a well-established robust estimator of scale [51, 45, 26]. The MAD is rescaled by the constant 1.4826 to ensure consistency with the standard deviation under normality. This choice reduces the risk that isolated spikes or noise-driven fluctuations are misinterpreted as meaningful adoption events.

Our qualitative analysis relies on an LLM-based filtering stage to reduce the volume of pull request comments requiring manual inspection, which constitutes a potential threat to construct validity. Although large language models are effective in semantic filtering tasks, their behavior is not deterministic and may lead to false negatives, particularly when relevant discussions are implicit, brief, or highly context-dependent. Importantly, the goal of this study is not to construct or evaluate a robust classification model for source code rejuvenation discussions. Instead, we use the LLM as a pragmatic tool to reduce manual effort while preserving semantically rich discussions that explicitly address motivations, challenges, or benefits of modernizing legacy Python code. Our manual validation of a random sample of rejected comments indicates that only a small fraction were potential false negatives, suggesting that the filtering did not systematically bias the qualitative findings. Nevertheless, some relevant discussions may have been excluded, and our results should be interpreted as capturing prominent and explicit rejuvenation-related rationales rather than an exhaustive set of all possible mentions.

Regarding external validity, our analysis involved 424 out of 343,878 Python projects (approximately 0.12%) hosted on GitHub. The selected projects fulfilled specific criteria: they are actively maintained, have at least a decade-long development history, and regularly integrate modern language features. However, this limited dataset may not comprehensively represent the broader Python community, restricting the generalizability of our results. Additionally, because our study focused exclusively on open-source repositories, our conclusions may not extend directly to industry or proprietary software development contexts.

For our qualitative investigation, we applied thematic coding to analyze pull-request comments relevant to Python source code rejuvenation. While our coding process followed established guidelines [49] and involved multiple authors to reduce subjectivity, the personal experience and perspectives of the researchers may still have influenced code interpretation and categorization. To mitigate this risk, we adopted a multi-cycle review process in which different authors independently coded the data, compared results, and resolved disagreements through discussion and consensus, with the participation of an additional author when needed. We also based our procedures on approaches validated in prior empirical studies on code review analysis [1, 48, 39]. Furthermore, we combined automated filtering—via a curated keyword search and large language model–based classification—with manual validation, allowing us to handle large volumes of comments while ensuring that the final dataset reflected to motivations and challenges for rejuvenation efforts. These measures aimed to enhance the reliability and validity of our findings, although we acknowledge that some bias and interpretive limitations may remain.

9. Conclusion

This study investigated how modern Python language features are adopted in open-source software projects. In particular, we examined when Python developers begin using new language features, how their adoption evolves over time, and which motivations and challenges influence source code rejuvenation in practice—especially in the period following the end-of-life of Python 2.7.

To this end, we employed a mixed-methods approach. First, we conducted a large-scale quantitative analysis of 424 open-source Python projects spanning 16 years of evolution (2008–2024), using an AST-based mining tool to track

feature usage at the file level over time and applying offline change-point detection to identify the onset of sustained adoption. Second, we complemented this analysis with a qualitative study of pull request discussions, systematically examining developers' comments to uncover explicit motivations and impediments associated with adopting modern Python features.

Our results show that adoption is a gradual and heterogeneous process. Quantitatively, developers often begin experimenting with new features shortly after their release, but widespread and sustained adoption typically takes several years to emerge. Features that offer lightweight, syntactic improvements tend to diffuse more rapidly, whereas constructs that introduce new paradigms or require broader structural changes exhibit longer maturation periods. Qualitatively, developers primarily associate code rejuvenation with improving code comprehension and consistency, while frequently citing compatibility constraints, maintenance costs, and learning effort as challenges that delay or limit adoption.

These findings contribute to Software Engineering research by offering a long-term empirical view of Python feature usage and by connecting longitudinal adoption patterns with themes observed in developers' pull request discussions. From a practical perspective, our results suggest considerations that may help tool builders and maintainers prioritize support for feature migration and modernization, for example by focusing on constructs with slower uptake and by accounting for commonly reported constraints such as version compatibility and perceived maintenance cost. For practitioners, this study provides empirical context that can support more informed planning of modernization efforts, including setting realistic expectations about adoption timelines and aligning feature uptake with project constraints and developer familiarity.

References

[1] Bacchelli, A., Bird, C., 2013. Expectations, outcomes, and challenges of modern code review, in: Notkin, D., Cheng, B.H.C., Pohl, K. (Eds.), 35th International Conference on Software Engineering, ICSE '13, San Francisco, CA, USA, May 18-26, 2013, IEEE Computer Society. pp. 712–721. URL: https://doi.org/10.1109/ICSE.2013.6606617, doi:10.1109/ICSE.2013.6606617.

[2] Biswas, S., Islam, M.J., Huang, Y., Rajan, H., 2019. Boa meets python: a boa dataset of data science software in python language, in: Storey, M.D., Adams, B., Haiduc, S. (Eds.), Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26-27 May 2019, Montreal, Canada, IEEE / ACM. pp. 577–581. URL: https://doi.org/10.1109/MSR.2019.00086, doi:10.1109/MSR.2019.00086.

[3] Bogachenkova, V., Nguyen, L., Ebert, F., Serebrenik, A., Castor, F., 2022. Evaluating atoms of confusion in the context of code reviews, in: IEEE International Conference on Software Maintenance and Evolution, ICSME 2022, Limassol, Cyprus, October 3-7, 2022, IEEE. pp. 404–408. URL: https://doi.org/10.1109/ICSME55016.2022.00048, doi:10.1109/ICSME55016.2022.00048.

[4] Callaú, O., Robbes, R., Tanter, É., Röthlisberger, D., 2011. How developers use the dynamic features of programming languages: the case of smalltalk, in: van Deursen, A., Xie, T., Zimmermann, T. (Eds.), Proceedings of the 8th International Working Conference on Mining Software Repositories, MSR 2011 (Co-located with ICSE), Waikiki, Honolulu, HI, USA, May 21-28, 2011, Proceedings, ACM. pp. 23–32. URL: https://doi.org/10.1145/1985441.1985448, doi:10.1145/1985441.1985448.

[5] Clarke, V., Braun, V., 2017. Thematic analysis. The journal of positive psychology 12, 297–298.

[6] Dabic, O., Aghajani, E., Bavota, G., 2021. Sampling projects in github for MSR studies, in: 18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021, IEEE. pp. 560–564.

[7] Dantas, R., Carvalho, A., Marcilio, D., Fantin, L., Silva, U., Lucas, W., Bonifácio, R., 2018. Reconciling the past and the present: An empirical study on the application of source code transformations to automatically rejuvenate java programs, in: Oliveto, R., Penta, M.D., Shepherd, D.C. (Eds.), 25th International Conference on Software Analysis, Evolution and Reengineering, SANER 2018, Campobasso, Italy, March 20-23, 2018, IEEE Computer Society. pp. 497–501. URL: https://doi.org/10.1109/SANER.2018.8330247, doi:10.1109/SANER.2018.8330247.

[8] Dietrich, J., Jezek, K., Brada, P., 2016. What java developers know about compatibility, and why this matters. Empir. Softw. Eng. 21, 1371–1396. URL: https://doi.org/10.1007/s10664-015-9389-1, doi:10.1007/S10664-015-9389-1.

[9] Foundation, P.S., . Python enhancement proposals (peps). https://peps.python.org/.

[10] Foundation, P.S., 2008. What's new in python 3.0. https://docs.python.org/3/whatsnew/3.0.html.

[11] Foundation, P.S., 2009. What's new in python 3.1. https://docs.python.org/3/whatsnew/3.1.html.

[12] Foundation, P.S., 2011. What's new in python 3.2. https://docs.python.org/3/whatsnew/3.2.html.

[13] Foundation, P.S., 2012. What's new in python 3.3. https://docs.python.org/3/whatsnew/3.3.html.

[14] Foundation, P.S., 2014. What's new in python 3.4. https://docs.python.org/3/whatsnew/3.4.html.

[15] Foundation, P.S., 2015. What's new in python 3.5. https://docs.python.org/3/whatsnew/3.5.html.

[16] Foundation, P.S., 2016. What's new in python 3.6. https://docs.python.org/3/whatsnew/3.6.html.

[17] Foundation, P.S., 2018. What's new in python 3.7. https://docs.python.org/3/whatsnew/3.7.html.

[18] Foundation, P.S., 2019. What's new in python 3.8. https://docs.python.org/3/whatsnew/3.8.html.

[19] Foundation, P.S., 2020. What's new in python 3.9. https://docs.python.org/3/whatsnew/3.9.html.

[20] Foundation, P.S., 2021. What's new in python 3.10. https://docs.python.org/3/whatsnew/3.10.html.

[21] Foundation, P.S., 2022. What's new in python 3.11. https://docs.python.org/3/whatsnew/3.11.html.

[22] Foundation, P.S., 2023. What's new in python 3.12. https://docs.python.org/3/whatsnew/3.12.html.

[23] Foundation, P.S., 2024a. History of python. https://docs.python.org/3/license.html#.

[24] Foundation, P.S., 2024b. Python (programming language). https://alalqab.com/en/Python_(programming_language).

[25] Grazia, L.D., Pradel, M., 2022. The evolution of type annotations in python: an empirical study, in: Roychoudhury, A., Cadar, C., Kim, M. (Eds.), Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022, ACM. pp. 209–220. URL: https://doi.org/10.1145/3540250.3549114, doi:10.1145/3540250.3549114.

[26] Hollander, M., Wolfe, D.A., Chicken, E., 2013. Nonparametric statistical methods. John Wiley & Sons.

[27] Hu, M., Zhang, Y., 2020. The python/c API: evolution, usage statistics, and bug patterns, in: Kontogiannis, K., Khomh, F., Chatzigeorgiou, A., Fokaefs, M., Zhou, M. (Eds.), 27th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2020, London, ON, Canada, February 18-21, 2020, IEEE. pp. 532–536. URL: https://doi.org/10.1109/SANER48275.2020.9054835, doi:10.1109/SANER48275.2020.9054835.

[28] Kapil, S., 2019. Clean Python: Elegant Coding in Python. First ed., Apress, New York, NY.

[29] Khatchadourian, R., Masuhara, H., 2018. Proactive empirical assessment of new language feature adoption via automated refactoring: The case of java 8 default methods. Art Sci. Eng. Program. 2, 6. URL: https://doi.org/10.22152/programming-journal.org/2018/2/6, doi:10.22152/PROGRAMMING-JOURNAL.ORG/2018/2/6.

[30] Kohn, T., van Rossum, G., II, G.B.B., Talin, Levkivskyi, I., 2020. Dynamic pattern matching with python, in: Flat, M. (Ed.), DLS 2020: Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages, Virtual Event, USA, November 17, 2020, ACM. pp. 85–98. URL: https://doi.org/10.1145/3426422.3426983, doi:10.1145/3426422.3426983.

[31] Lin, H.Y., Thongtanunam, P., 2023. Towards automated code reviews: Does learning code structure help?, in: Zhang, T., Xia, X., Novielli, N. (Eds.), IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21-24, 2023, IEEE. pp. 703–707. URL: https://doi.org/10.1109/SANER56733.2023.00075, doi:10.1109/SANER56733.2023.00075.

[32] Lindstrom, G., 2005. Programming with python. IT Prof. 7, 10–16. URL: https://doi.org/10.1109/MITP.2005.120, doi:10.1109/MITP.2005.120.

[33] Lucas, W., Carvalho, F., Nunes, R.C., Bonifácio, R., Saraiva, J., Accioly, P.R.G., 2024. Embracing modern C++ features: An empirical assessment on the KDE community. J. Softw. Evol. Process. 36. URL: https://doi.org/10.1002/smr.2605, doi:10.1002/SMR.2605.

[34] Lucas, W., Nunes, R.C., Bonifácio, R., Carvalho, F., Lima, R., Silva, M., Torres, A., Accioly, P.R.G., Monteiro, E., Saraiva, J., 2025. Understanding the adoption of modern javascript features: An empirical study on open-source systems. Empir. Softw. Eng. 30, 107. URL: https://doi.org/10.1007/s10664-025-10663-9, doi:10.1007/S10664-025-10663-9.

[35] Luo, X., Xue, Y., Xing, Z., Sun, J., 2022. PRCBERT: prompt learning for requirement classification using bert-based pretrained language models, in: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022, ACM. pp. 75:1–75:13. URL: https://doi.org/10.1145/3551349.3560417, doi:10.1145/3551349.3560417.

[36] Malloy, B.A., Power, J.F., 2017. Quantifying the transition from python 2 to 3: An empirical study of python applications, in: Bener, A., Turhan, B., Biffl, S. (Eds.), 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2017, Toronto, ON, Canada, November 9-10, 2017, IEEE Computer Society. pp. 314–323. URL: https://doi.org/10.1109/ESEM.2017.45, doi:10.1109/ESEM.2017.45.

[37] Malloy, B.A., Power, J.F., 2019. An empirical analysis of the transition from python 2 to python 3. Empir. Softw. Eng. 24, 751–778. URL: https://doi.org/10.1007/s10664-018-9637-2, doi:10.1007/S10664-018-9637-2.

[38] Mostafa, S., Rodriguez, R., Wang, X., 2017. Experience paper: a study on behavioral backward incompatibilities of java software libraries, in: Bultan, T., Sen, K. (Eds.), Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, Santa Barbara, CA, USA, July 10 - 14, 2017, ACM. pp. 215–225. URL: https://doi.org/10.1145/3092703.3092721, doi:10.1145/3092703.3092721.

[39] Oliveira, D., 2021. Recommending code understandability improvements based on code reviews, in: 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021 - Workshops, Melbourne, Australia, November 15-19, 2021, IEEE. pp. 131–132. URL: https://doi.org/10.1109/ASEW52652.2021.00035, doi:10.1109/ASEW52652.2021.00035.

[40] Parnin, C., Bird, C., Murphy-Hill, E.R., 2011. Java generics adoption: how new features are introduced, championed, or ignored, in: van Deursen, A., Xie, T., Zimmermann, T. (Eds.), Proceedings of the 8th International Working Conference on Mining Software Repositories, MSR 2011 (Co-located with ICSE), Waikiki, Honolulu, HI, USA, May 21-28, 2011, Proceedings, ACM. pp. 3–12. URL: https://doi.org/10.1145/1985441.1985446, doi:10.1145/1985441.1985446.

[41] Peng, Y., Zhang, Y., Hu, M., 2021. An empirical study for common language features used in python projects, in: 28th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2021, Honolulu, HI, USA, March 9-12, 2021, IEEE. pp. 24–35. URL: https://doi.org/10.1109/SANER50967.2021.00012, doi:10.1109/SANER50967.2021.00012.

[42] Pirkelbauer, P., Dechev, D., Stroustrup, B., 2010. Source code rejuvenation is not refactoring, in: van Leeuwen, J., Muscholl, A., Peleg, D., Pokorný, J., Rumpe, B. (Eds.), SOFSEM 2010: Theory and Practice of Computer Science, 36th Conference on Current Trends in Theory and Practice of Computer Science, Spindleruv Mlýn, Czech Republic, January 23-29, 2010. Proceedings, Springer. pp. 639–650. URL: https://doi.org/10.1007/978-3-642-11266-9_53, doi:10.1007/978-3-642-11266-9_53.

[43] Rak-amnouykit, I., McCrevan, D., Milanova, A.L., Hirzel, M., Dolby, J., 2020. Python 3 types in the wild: a tale of two type systems, in: Flat, M. (Ed.), DLS 2020: Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages, Virtual Event, USA, November 17, 2020, ACM. pp. 57–70. URL: https://doi.org/10.1145/3426422.3426981, doi:10.1145/3426422.3426981.

[44] Review, D.b., 2026. "it makes the code clearer": Why developers adopt modern python features in open source projects. URL: https://doi.org/10.5281/zenodo.18462376, doi:10.5281/zenodo.18462376.

[45] Ronchetti, E.M., Huber, P.J., 2009. Robust statistics. John Wiley & Sons Hoboken, NJ, USA.

[46] Rosenkranz, S., Staegemann, D., Volk, M., Turowski, K., 2024. Explaining the business-technological age of legacy information systems. IEEE Access 12, 84579–84611. URL: https://doi.org/10.1109/ACCESS.2024.3414377, doi:10.1109/ACCESS.2024.3414377.

[47] Rózsa, B., Antal, G., Ferenc, R., 2022. Don't DIY: automatically transform legacy python code to support structural pattern matching, in: 22nd IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2021, Limassol, Cyprus, October 3, 2022, IEEE. pp. 164–169. URL: https://doi.org/10.1109/SCAM55253.2022.00024, doi:10.1109/SCAM55253.2022.00024.

[48] Sadowski, C., Söderberg, E., Church, L., Sipko, M., Bacchelli, A., 2018. Modern code review: a case study at google, in: Paulisch, F., Bosch, J. (Eds.), Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2018, Gothenburg, Sweden, May 27 - June 03, 2018, ACM. pp. 181–190. URL: https://doi.org/10.1145/3183519.3183525, doi:10.1145/3183519.3183525.

[49] Saldana, J., 2012. The Coding Manual for Qualitative Researchers. English short title catalogue Eighteenth Century collection, SAGE Publications. URL: https://books.google.com.br/books?id=kUms8QrE_SAC.

[50] Scarsbrook, J.D., Utting, M., Ko, R.K.L., 2023. Typescript's evolution: An analysis of feature adoption over time, in: 20th IEEE/ACM International Conference on Mining Software Repositories, MSR 2023, Melbourne, Australia, May 15-16, 2023, IEEE. pp. 109–114. URL: https://doi.org/10.1109/MSR59073.2023.00027, doi:10.1109/MSR59073.2023.00027.

[51] Stuart, M., 1984. Understanding robust and exploratory data analysis.

[52] Sun, X., Li, X., Li, J., Wu, F., Guo, S., Zhang, T., Wang, G., 2023. Text classification via large language models, in: Bouamor, H., Pino, J., Bali, K. (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023, Association for Computational Linguistics. pp. 8990–9005. URL: https://doi.org/10.18653/v1/2023.findings-emnlp.603, doi:10.18653/V1/2023.FINDINGS-EMNLP.603.

[53] Truong, C., Oudre, L., Vayatis, N., 2020. Selective review of offline change point detection methods. Signal Processing 167, 107299. URL: https://www.sciencedirect.com/science/article/pii/S0165168419303494, doi:https://doi.org/10.1016/j.sigpro.2019.107299.

[54] Vándor, N., Antal, G., Hegedüs, P., Ferenc, R., 2024. On the usefulness of python structural pattern matching: An empirical study, in: IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2024, Rovaniemi, Finland, March 12-15, 2024, IEEE. pp. 501–511. URL: https://doi.org/10.1109/SANER60148.2024.00058, doi:10.1109/SANER60148.2024.00058.

[55] Zhang, Z., Xing, Z., Xia, X., Xu, X., Zhu, L., 2022. Making python code idiomatic by automatic refactoring non-idiomatic python code with pythonic idioms, in: Roychoudhury, A., Cadar, C., Kim, M. (Eds.), Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022, ACM. pp. 696–708. URL: https://doi.org/10.1145/3540250.3549143, doi:10.1145/3540250.3549143.

[56] Zhao, Y., Serebrenik, A., Zhou, Y., Filkov, V., Vasilescu, B., 2017. The impact of continuous integration on other software development practices: a large-scale empirical study, in: Rosu, G., Penta, M.D., Nguyen, T.N. (Eds.), Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering, ASE 2017, Urbana, IL, USA, October 30 - November 03, 2017, IEEE Computer Society. pp. 60–71. URL: https://doi.org/10.1109/ASE.2017.8115619, doi:10.1109/ASE.2017.8115619.